We can also get a single attribute. For example, this listing shows a request that fetches the number of shards.

Listing 6.9 Fetching a single attribute

```
GET cars_with_custom_settings/_settings/index.number_of_shards
```

Here, the properties are enclosed in an inner object `index`, so we must prefix the attribute with the top-level object like this: `index.<attribute_name>`.

6.2.4 Indexes with mappings

In addition to settings, we can provide field mappings when creating an index. This is how we create a schema for our data model. The following listing shows the mechanism to create an index with a mapping definition for a `car` type (it's actually `cars_with_mappings` index, but for brevity, we can assume the index considers `car` entities) with `make`, `model`, and `registration_year` attributes.

Listing 6.10 Creating an index with field mappings for a `car` document

```
PUT cars_with_mappings
{
  "mappings": {
    "properties": {
      "make":{
        "type": "text"
      },
      "model":{
        "type": "text"
      },
      "registration_year":{
        "type": "date",
        "format": "dd-MM-yyyy"
      }
    }
  }
}
```

The mappings object encloses the properties.

The fields with car data types are declared here.

Declares make as a text type

Declares registration_year as a date type

The field's custom format

Of course, we can also combine settings and mappings. The next listing shows this approach.

Listing 6.11 Creating an index with both settings and mappings

```
PUT cars_with_settings_and_mappings
{
  "settings": {
    "number_of_replicas": 3
  },
  "mappings": {
    "properties": {
      "make":{
```

Index with both settings and mappings

Settings on the index

Mappings schema definition